



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Angela Jimena Santizo Escobar

Carné: 1207925

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
 - a. Es una interfaz que permite a dos sistemas comunicarse por HTTP siguiendo el estilo arquitectónico REST. Los datos se tratan como recursos identificados por una URL. Se manipulan con métodos HTTP estándar y el servidor no guarda estado entre solicitudes.
2. ¿Qué es un endpoint?
 - a. Es una dirección concreta de la API donde se puede acceder a un recurso u operación.
3. ¿Qué diferencia existe entre una URL y un endpoint?
 - a. a URL es la dirección completa. El endpoint es la combinación de una ruta y un método HTTP que ofrece una operación. Una misma URL puede tener varios endpoints según el método.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
 - a. GET: obtiene información sin modificarla.
 - b. POST: crea un recurso nuevo.
 - c. PUT: reemplaza o actualiza un recurso completo.
 - d. PATCH: modifica solo algunos campos.
 - e. DELETE: elimina un recurso.
5. ¿Qué es JSON y por qué es común en las APIs?
 - a. JavaScript Object Notation es un formato de texto ligero basado en pares clave-valor, arreglos y objetos. Es común porque es legible para personas, fácil de generar y procesar en casi cualquier lenguaje, y ocupa menos espacio que XML.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
 - a. es un número de tres dígitos con el que el servidor indica el resultado de la solicitud.
Ejemplos:
 - 200 OK: la solicitud fue exitosa.
 - 404 Not Found: el recurso no existe.
 - 500 Internal Server Error: falló algo interno del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
 - a. El código de estado, los encabezados, el cuerpo con los datos en JSON o un mensaje de error, y a veces el tiempo de respuesta y el tamaño.

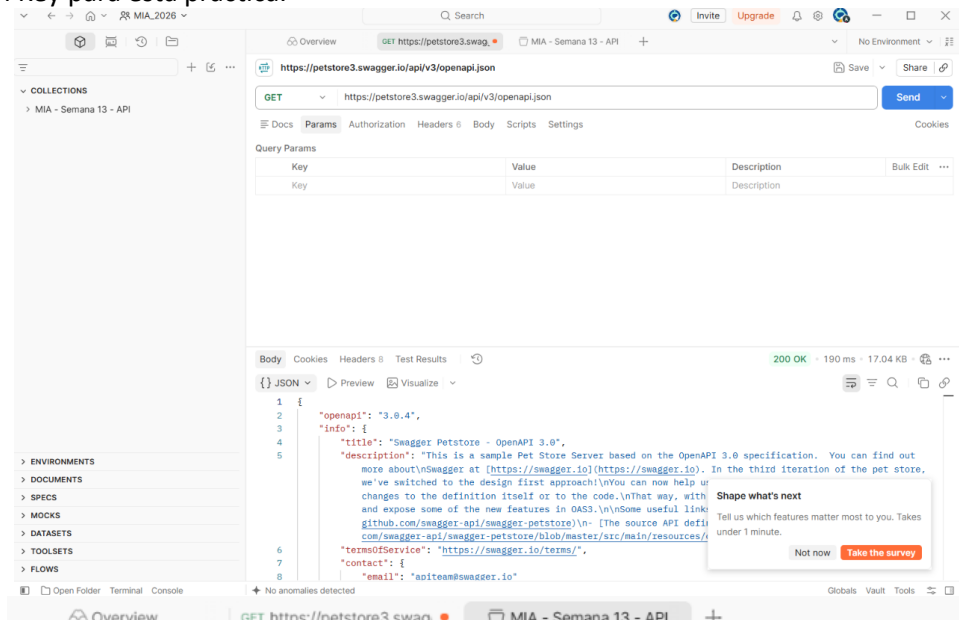
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
 - a. Permite probar endpoints sin escribir código, validar métodos, parámetros, headers y body, organizar solicitudes en colecciones, reutilizar variables, detectar errores temprano y compartir pruebas con el equipo.
9. ¿Qué es y para qué sirve Swagger UI?
 - a. Es una interfaz web que muestra la documentación de una API (según la especificación OpenAPI) y permite probar los endpoints directamente con "Try it out".
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
 - a. Es una herramienta para diseñar, probar, documentar y automatizar solicitudes a APIs, con colecciones, entornos, variables y scripts de prueba.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.



The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/openapi.json`. The response is a JSON object with the following structure:

```

{
  "openapi": "3.0.4",
  "info": {
    "title": "Swagger Petstore - OpenAPI 3.0",
    "description": "This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about Swagger at (https://swagger.io)(https://swagger.io). In the third iteration of the pet store, we've switched to the design first approach! You can now help us shape what's next for Swagger by telling us which features matter most to you. Takes under 1 minute.",
    "termsOfService": "https://swagger.io/terms/",
    "contact": {
      "email": "apiteam@swagger.io"
    }
  }
}

```

Below the response, the 'Variables' tab is active, showing a table of environment variables:

Variable	Value
baseUrl	https://petstore3.swagger.io/api/v3
Add variable	

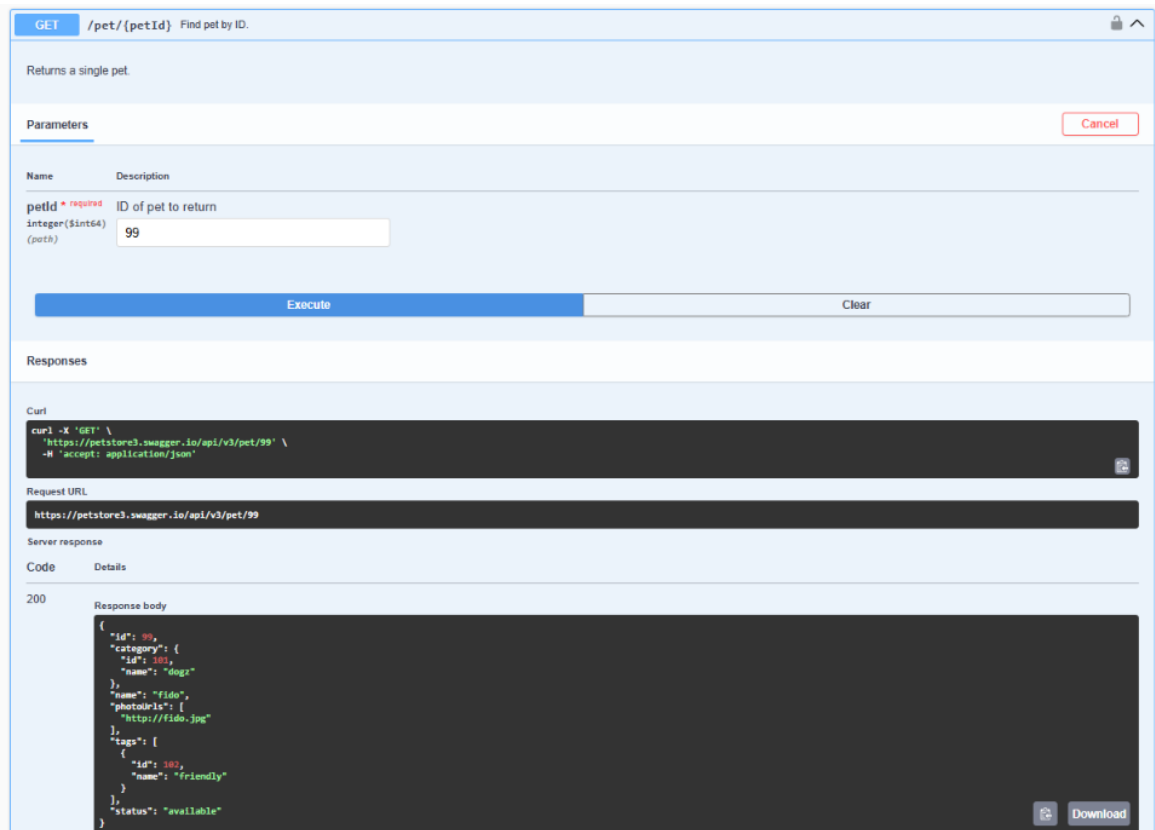
2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
 - GET /pet/{petId}
- Indique el método HTTP utilizado.
 - GET
- Indique qué parámetro necesita la solicitud.
 - petId, un entero que se envía en la ruta (path) y es obligatorio. Valor usado: 99.

- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.
 - La API devolvió un objeto JSON con los datos de la mascota con ID 99: su identificador (id: 99), su categoría (category, con id 101 y nombre "dogz"), su nombre ("fido"), la lista de fotos (photoUrls, con "http://fido.jpg"), una etiqueta (tags, con id 102 y nombre "friendly") y su estado (status: "available", es decir, disponible).

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST `/pet` para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

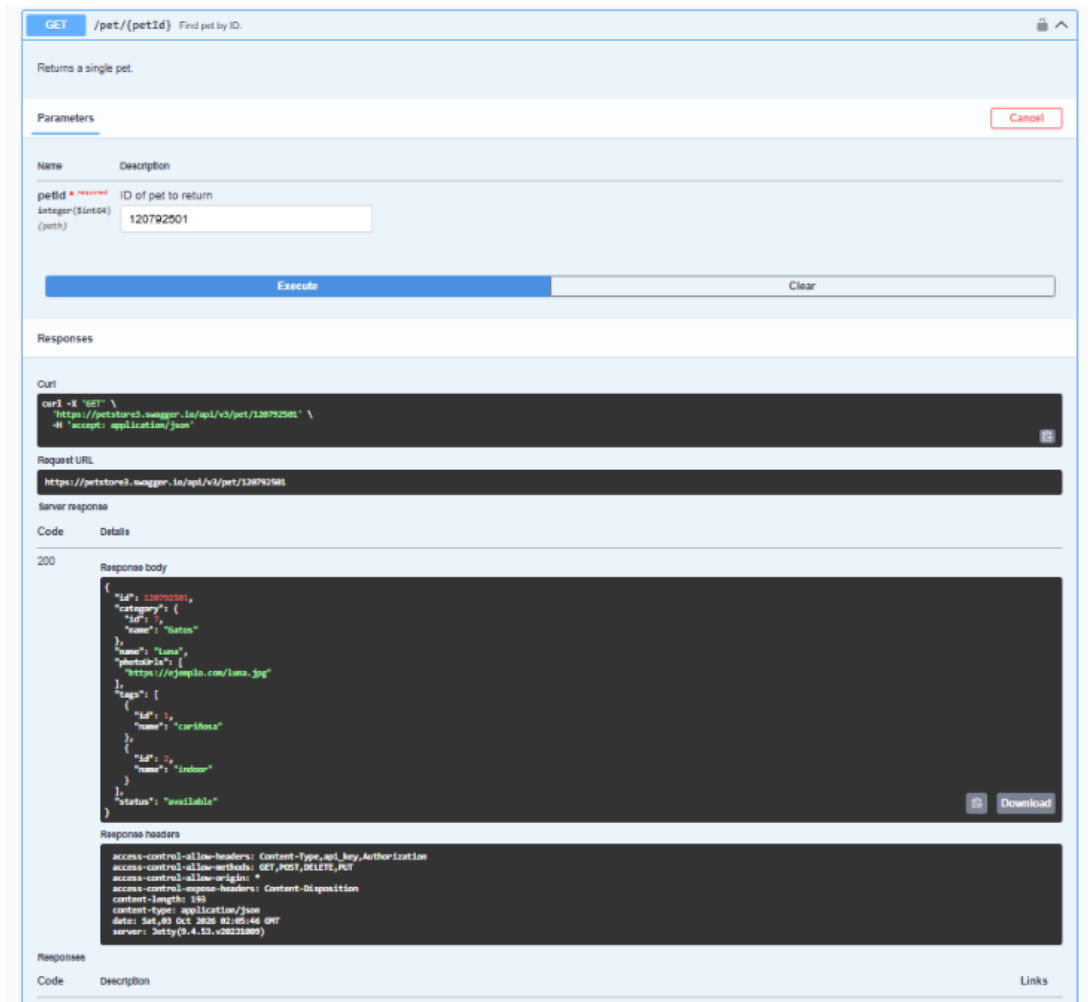
id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET `/pet/{petId}`.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

EVIDENCIAS

- POST/`pet`



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
 - PUT, porque actualiza un recurso que ya existe enviando su representación completa. POST crea uno nuevo.
- ¿Qué datos fueron enviados en el body?
 - El JSON completo con name cambiado a "Luna Actualizada" y status cambiado de available a sold.
- ¿Qué código HTTP devolvió la API?
 - Normalmente devuelve 200, como en este caso.
- ¿Cómo comprobó que el cambio realmente ocurrió?
 - Hice un GET /pet/120792501 y vi el nuevo nombre y estado.

EVIDENCIAS

- PUT/pet

PUT /pet Update an existing pet

Update an existing pet by id.

Parameters

No parameters

Request body

application/json

Update an existent pet in the store

Edit Value

Schema

```
{
  "id": 120792581,
  "name": "Luna Actualizada",
  "category": { "id": 7, "name": "Gatos" },
  "photoUrls": [ "https://example.com/luna.jpg" ],
  "tags": [
    { "id": 1, "name": "carilona" },
    { "id": 2, "name": "indoor" }
  ],
  "status": "sold"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 120792581,
    "name": "Luna Actualizada",
    "category": { "id": 7, "name": "Gatos" },
    "photoUrls": [ "https://example.com/luna.jpg" ],
    "tags": [
      { "id": 1, "name": "carilona" },
      { "id": 2, "name": "indoor" }
    ],
    "status": "sold"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code

Details

Code

Details

200

Response body

```
{
  "id": 120792581,
  "category": {
    "id": 7,
    "name": "Gatos"
  },
  "name": "Luna Actualizada",
  "photoUrls": [
    "https://example.com/luna.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "carilona"
    },
    {
      "id": 2,
      "name": "indoor"
    }
  ],
  "status": "sold"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api-key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 200
content-type: application/json
date: Sat, 08 Oct 2026 02:07:17 GMT
server: Jetty(9.4.53.v20211004)
```

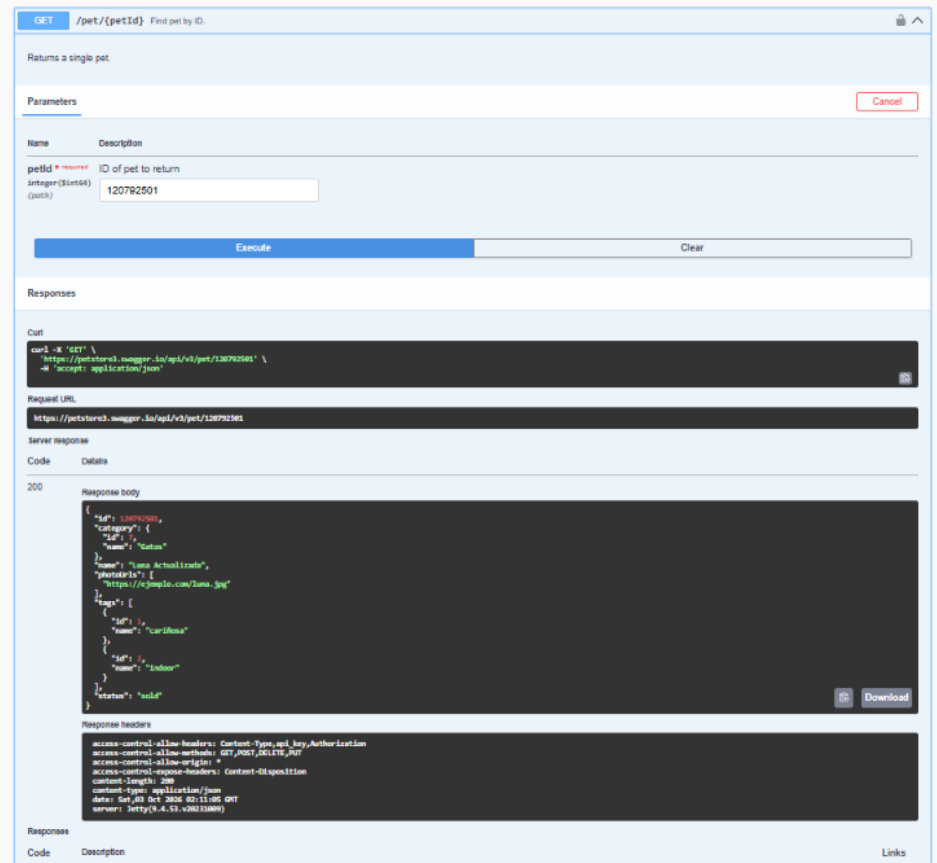
Responses

Code

Description

Links

- GET /pet/120792501

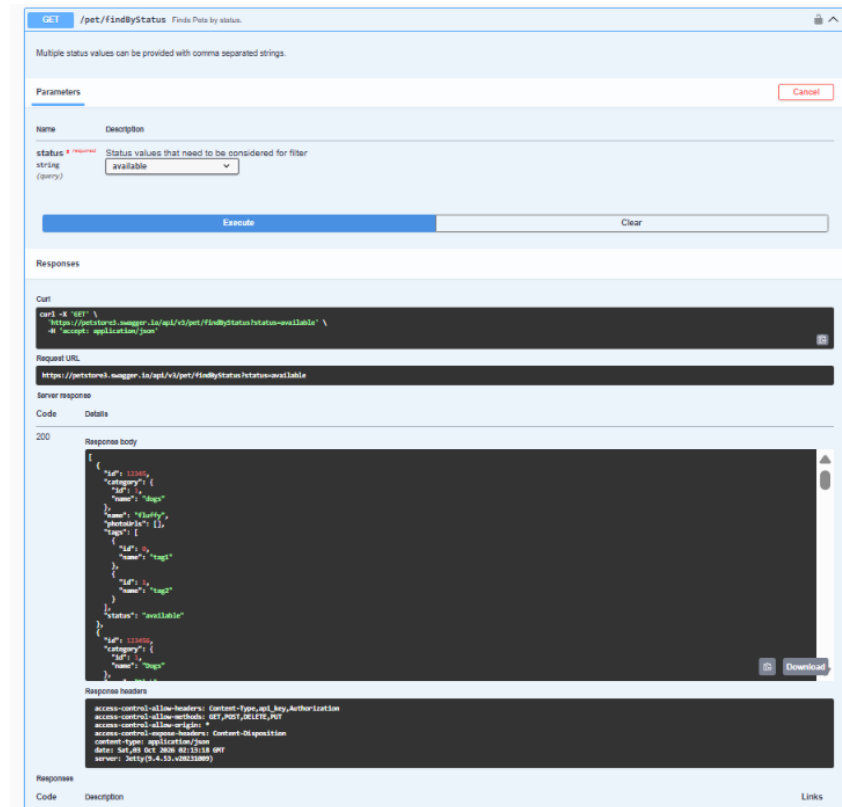


5. Consultar utilizando parámetros

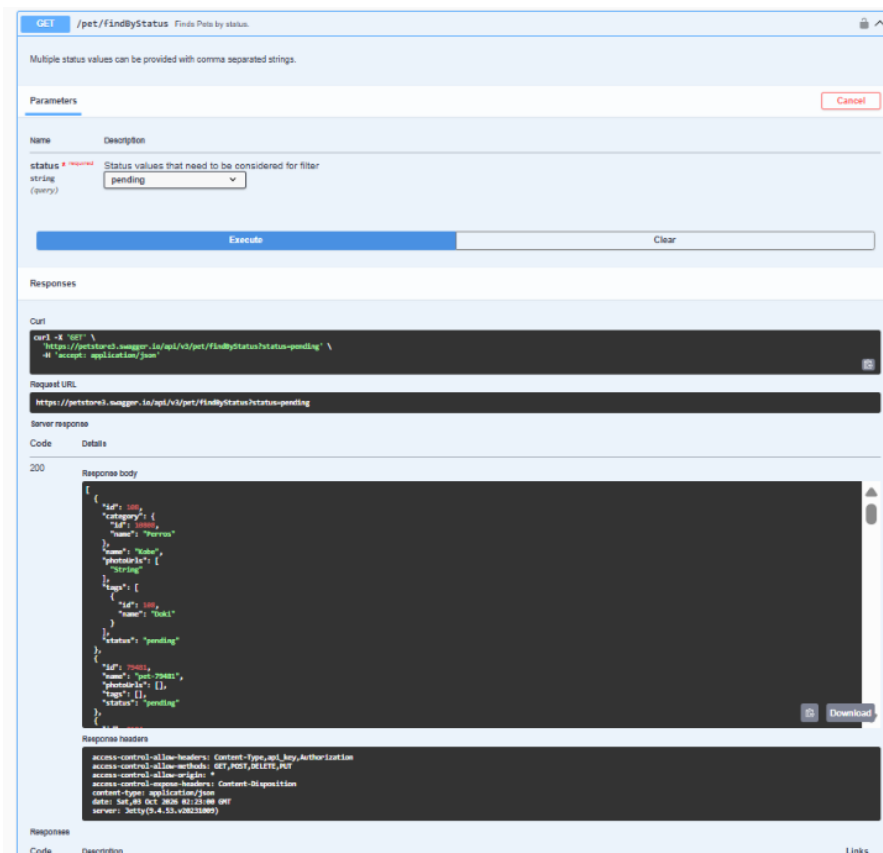
- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
 - status
- Explique dónde se envía ese parámetro.
 - En la query string de la URL: /pet/findByStatus?status=available.
- Observe la estructura de la respuesta.
 - un arreglo [...] de objetos mascota, cada uno con id, name, category, photoUrls, tags y status.
- Indique qué sucede si cambia el valor del parámetro por pending.
 - Al cambiar el valor de status de available a pending, la API sigue respondiendo con código 200, pero devuelve una lista distinta: ahora solo trae las mascotas cuyo estado es "pending" (pendientes o en proceso de venta). La URL de la solicitud cambia a /pet/findByStatus?status=pending y en la respuesta cada mascota muestra "status": "pending" (por ejemplo, "Kobe" y "pet-79481"). Esto demuestra que el parámetro funciona como un filtro: el endpoint es el mismo, pero el resultado depende del valor enviado.

EVIDENCIAS

- Available

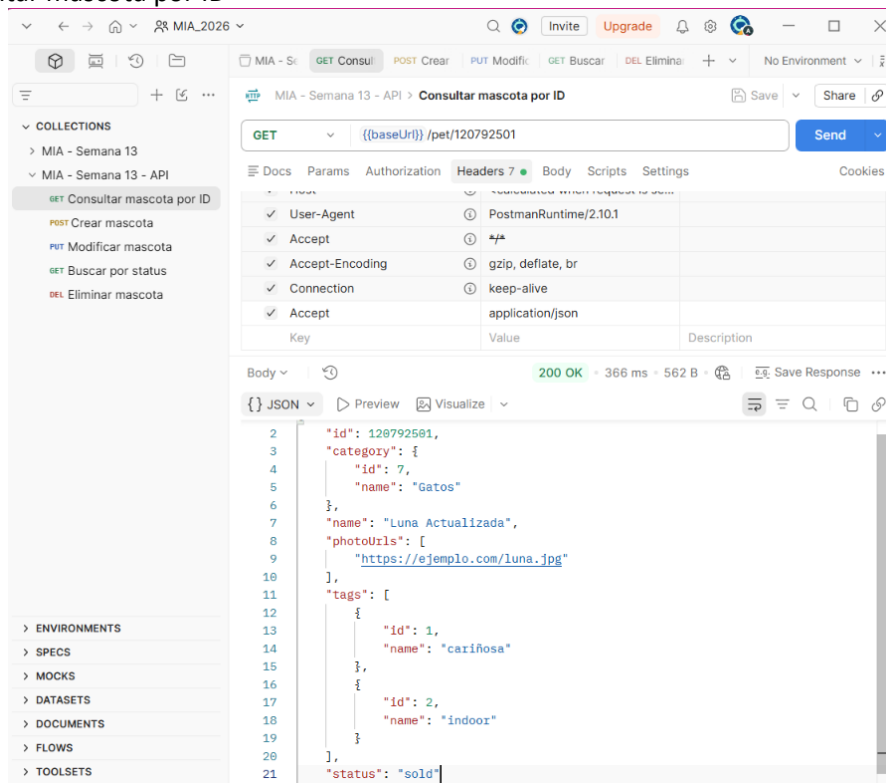


- Pending

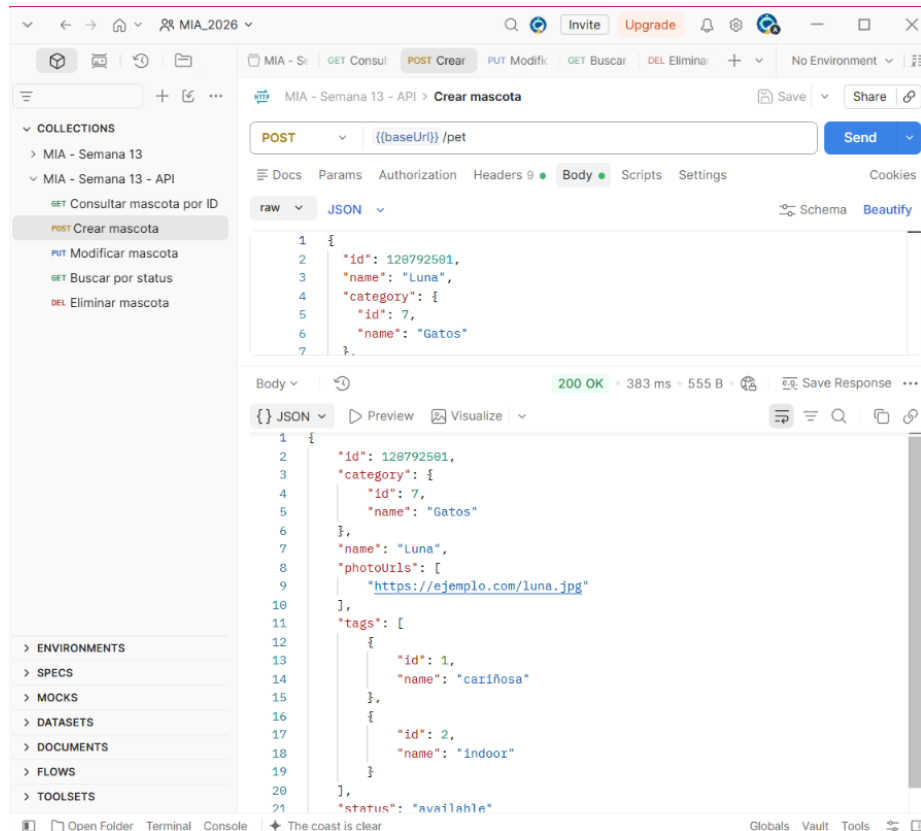


6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID



- POST – Crear mascota



- PUT – Modificar mascota

The screenshot shows the Postman interface for a PUT request. The URL is `{{baseUrl}}/pet`. The request body is a JSON object representing a pet with the following structure:

```

1 {
2   "id": 120792501,
3   "name": "Luna Actualizada",
4   "category": { "id": 7, "name": "Gatos" },
5   "photoUrls": ["https://ejemplo.com/luna.jpg"],
6   "tags": [
7     { "id": 1, "name": "carifiosa" }
8   ]
9 }

```

The response status is 200 OK, with a response time of 88 ms and a body size of 562 B. The response body is a JSON object with the following structure:

```

1 {
2   "id": 120792501,
3   "category": {
4     "id": 7,
5     "name": "Gatos"
6   },
7   "name": "Luna Actualizada",
8   "photoUrls": [
9     "https://ejemplo.com/luna.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "carifiosa"
15    },
16    {
17      "id": 2,
18      "name": "indoor"
19    }
20  ],
21  "status": "sold"
22 }

```

- GET – Buscar por status

The screenshot shows the Postman interface for a GET request. The URL is `{{baseUrl}}/pet/findByStatus?status=available`. The request parameters are defined in the Query Params section:

Key	Value	Description	Bulk Edit
status	available		

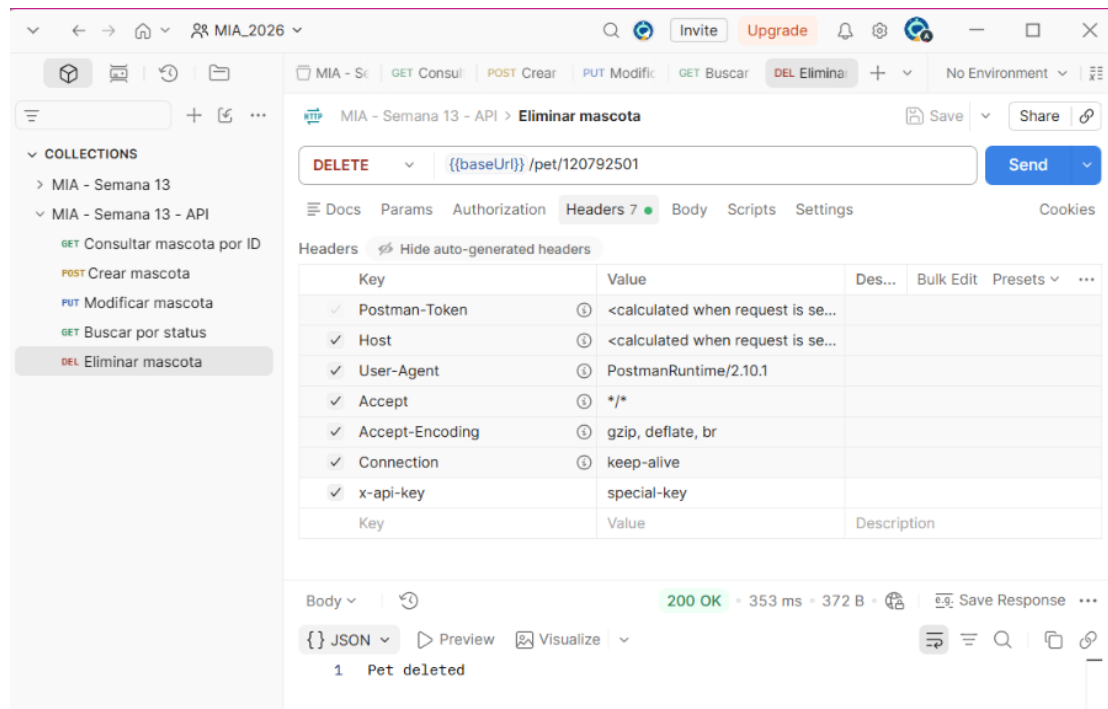
The response status is 200 OK, with a response time of 544 ms and a body size of 126.53 KB. The response body is a JSON array containing one pet object with the following structure:

```

1 [
2   {
3     "id": 12345,
4     "category": {
5       "id": 1,
6       "name": "dogs"
7     },
8     "name": "fluffy",
9     "photoUrls": [],
10    "tags": [
11      {
12        "id": 0,
13        "name": "tag1"
14      },
15      {
16        "id": 1,
17        "name": "tag2"
18      }
19    ],
20    "status": "available"
21  }
22 ]

```

- DELETE – Eliminar mascota



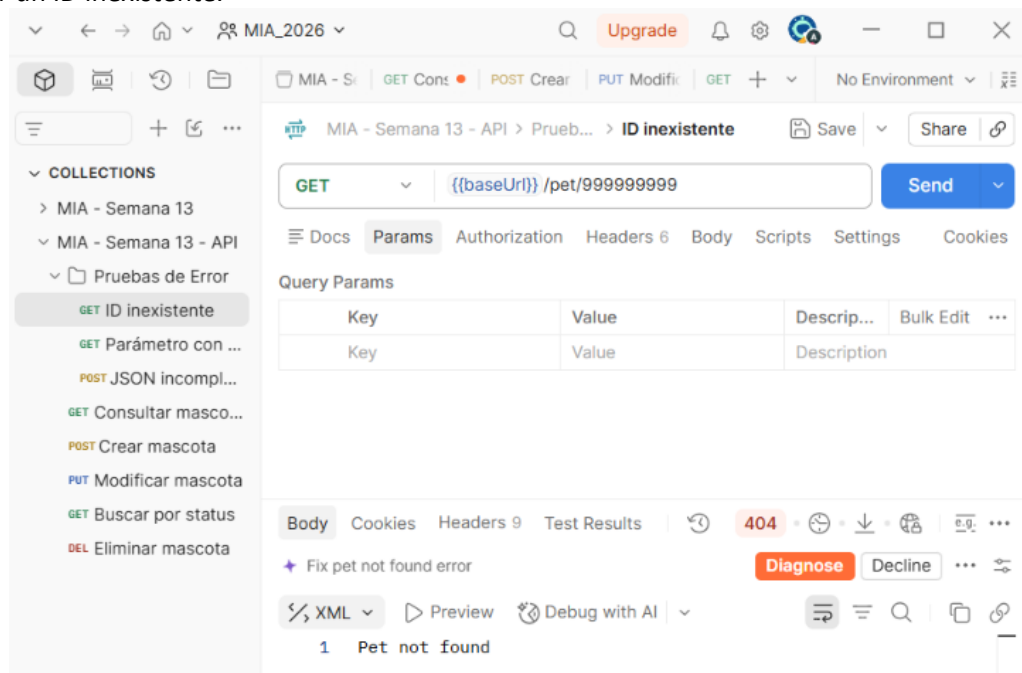
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.



a) Solicitud realizada: se ve el método GET y la URL `{{baseUrl}}/pet/999999999`.

b) Código HTTP recibido: `404` en la parte de la respuesta.

c) Respuesta de la API: Pet not found en el body.

d) Qué esperaba que ocurriera:

Esperaba que la API no encontrara ninguna mascota con ese ID y devolviera un error 404 indicando que el recurso no existe.

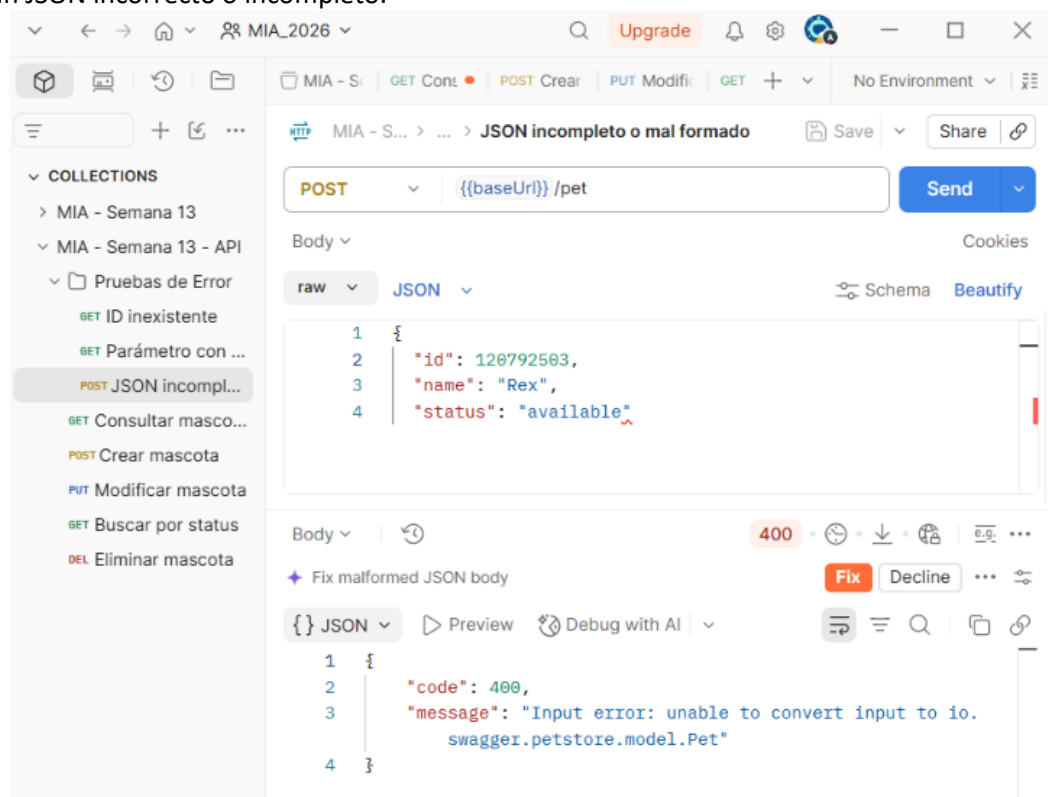
e) Qué ocurrió realmente:

Ocurrió lo esperado: la API respondió con código 404 y el mensaje "Pet not found". No devolvió datos de ninguna mascota.

f) Explicación de la causa:

El ID 999999999 no corresponde a ninguna mascota registrada en el servidor. Como el recurso solicitado no existe, la API responde con el código 404, que es el estándar HTTP para indicar que algo no fue encontrado. Este código permite a un programa identificar que el recurso no existe, sin tener que interpretar el texto del mensaje.

2. Enviar un JSON incorrecto o incompleto.



a) Solicitud realizada:

Método POST a `https://petstore3.swagger.io/api/v3/pet` desde Postman, con un body en formato JSON raw al que se le omitió a propósito la llave de cierre }:

json

```
{
  "id": 120792503,
  "name": "Rex",
  "status": "available"
```

b) Código HTTP recibido: 400 (Bad Request).

c) Respuesta de la API:

Un JSON con el código y el mensaje de error:

json

```
{
  "code": 400,
```

```
"message": "Input error: unable to convert input to io.swagger.petstore.model.Pet"
}
```

d) Qué esperaba que ocurriera:

Esperaba que la API rechazara la solicitud y devolviera un error 400, porque el JSON enviado está incompleto y el servidor no puede interpretarlo.

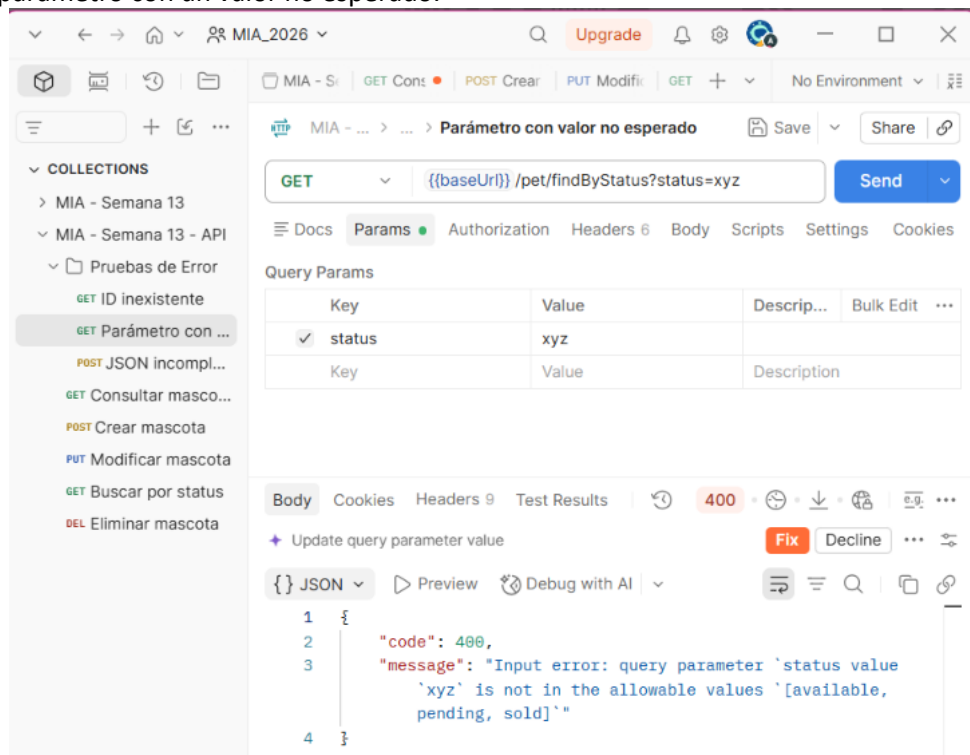
e) Qué ocurrió realmente:

Ocurrió lo esperado. La API no creó ninguna mascota y respondió con un 400 y un mensaje indicando que no pudo convertir la entrada al modelo Pet.

f) Explicación de la causa:

El cuerpo enviado no es un JSON válido porque le falta la llave de cierre }. El servidor intenta convertir el texto recibido en un objeto Pet, pero al estar mal formado no puede hacerlo y responde con 400, el código HTTP que indica que el error está en la solicitud del cliente y no en el servidor. Postman también marcó el error con un subrayado rojo en el editor, antes de enviar.

3. Utilizar un parámetro con un valor no esperado.



a) Solicitud realizada: Método GET a <https://petstore3.swagger.io/api/v3/pet/findByStatus?status=xyz> desde Postman, enviando el parámetro status con el valor xyz, que no es uno de los valores permitidos. No lleva body.

b) Código HTTP recibido: 400 (Bad Request).

c) Respuesta de la API:

Un JSON con el código y el mensaje de error:

```
json
{
  "code": 400,
  "message": "Input error: query parameter `status` value `xyz` is not in the allowable values `[available, pending, sold]`"
}
```

d) Qué esperaba que ocurriera:

Esperaba que la API rechazara la solicitud con un error 400, porque el parámetro status solo acepta los

valores available, pending y sold, y xyz no es ninguno de ellos.

e) Qué ocurrió realmente:

Ocurrió lo esperado. La API no devolvió ninguna lista de mascotas y respondió con un 400. El mensaje de error indica de forma clara cuál fue el valor inválido y cuáles son los valores permitidos.

f) Explicación de la causa:

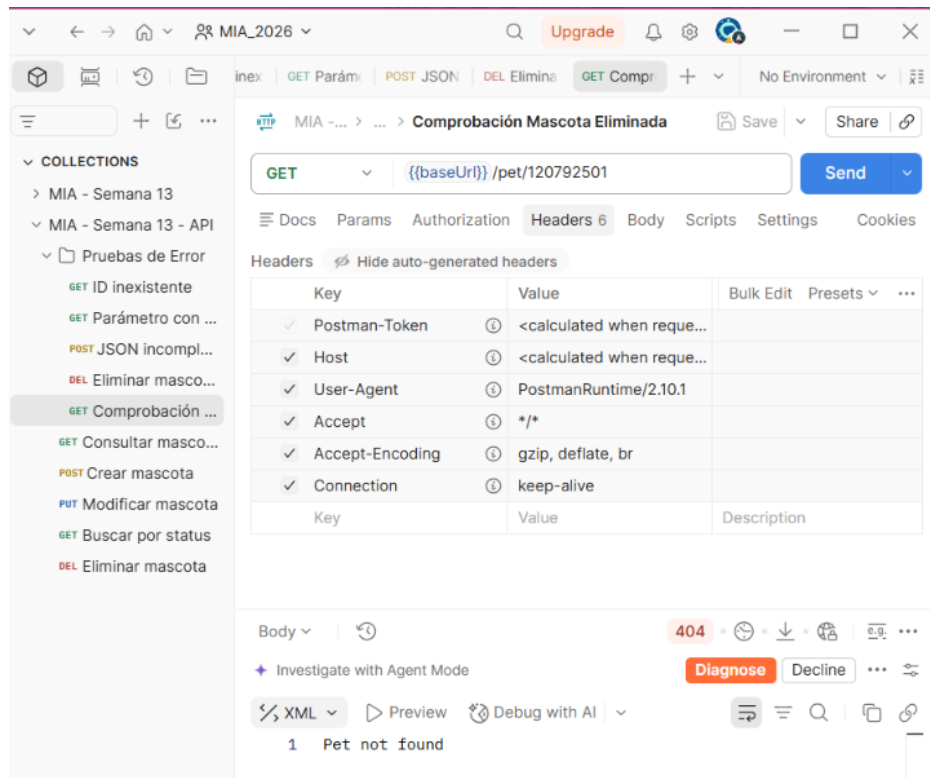
El valor xyz no pertenece al conjunto de valores aceptados por el parámetro status. La API valida el parámetro antes de procesar la consulta y, al no cumplirse la validación, responde con 400, el código HTTP que indica que el error está en la solicitud del cliente. El mensaje de la respuesta permite corregir el error, porque lista los valores válidos.

4. Intentar eliminar un recurso que ya no existe.

The screenshot shows the Postman interface for a DELETE request. The URL is `{{baseUrl}}/pet/120792501`. The request headers are as follows:

Key	Value	Bulk Edit	Presets
Postman-Token	<calculated when request is sent>		
Host	<calculated when request is sent>		
User-Agent	PostmanRuntime/2.10.1		
Accept	*/*		
Accept-Encoding	gzip, deflate, br		
Connection	keep-alive		
x-api-key	special-key		

The response status is 200, and the body is `Pet deleted`.



a) **Solicitud realizada:** Método DELETE a <https://petstore3.swagger.io/api/v3/pet/120792501> desde Postman, ejecutado por segunda vez sobre una mascota que ya había sido eliminada. No lleva body.

b) **Código HTTP recibido:** 200 (OK).

c) **Respuesta de la API:** Pet deleted.

d) **Qué esperaba que ocurriera:** Esperaba un error 404, porque la mascota con ID 120792501 ya no existía y no habría nada que eliminar.

e) **Qué ocurrió realmente:**

La API devolvió 200 con el mensaje "Pet deleted", como si hubiera eliminado algo. Para comprobar que la mascota no existía, hice un `GET /pet/120792501`, que devolvió **404** con el mensaje Pet not found.

f) **Explicación de la causa:**

La Petstore no verifica si el recurso existe antes de responder al DELETE, así que devuelve éxito aunque no haya nada que borrar (el DELETE es idempotente: repetirlo deja el sistema en el mismo estado). Por eso un programa no debe confiar solo en el código del DELETE para saber si el recurso existe, y debe confirmarlo con un GET, que sí devuelve 404.

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

1. POST: Registrar mascota

Postman interface showing a POST request to `Registar mascota` (API endpoint: `/pet`). The request body is a JSON object:

```
1 {
2   "id": 120792504,
3   "name": "Milo",
4   "category": { "id": 7, "name": "Gatos" },
5   "photoUrls": ["https://ejemplo.com/milo.jpg"],
6   "tags": [{ "id": 1, "name": "juguetón" }],
7   "status": "available"
8 }
```

The response is a 200 status code, and the response body is the same JSON object as the request body.

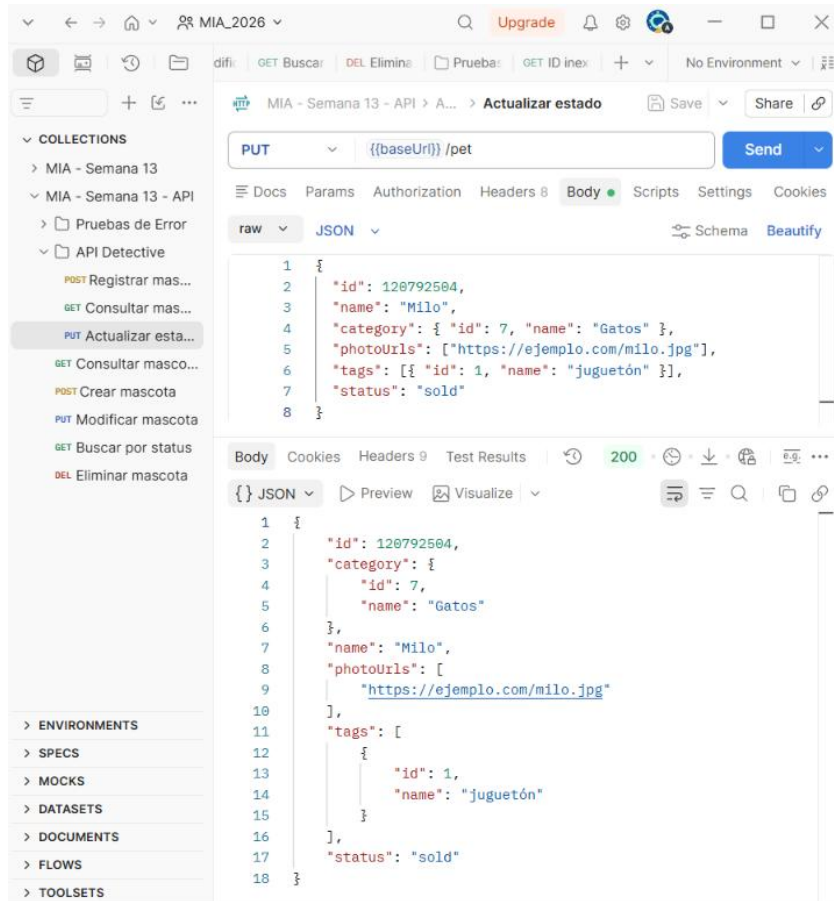
2. GET: Consultar mascota

Postman interface showing a GET request to `Consultar mascota` (API endpoint: `/pet/120792504`). The request is a simple GET request with no body.

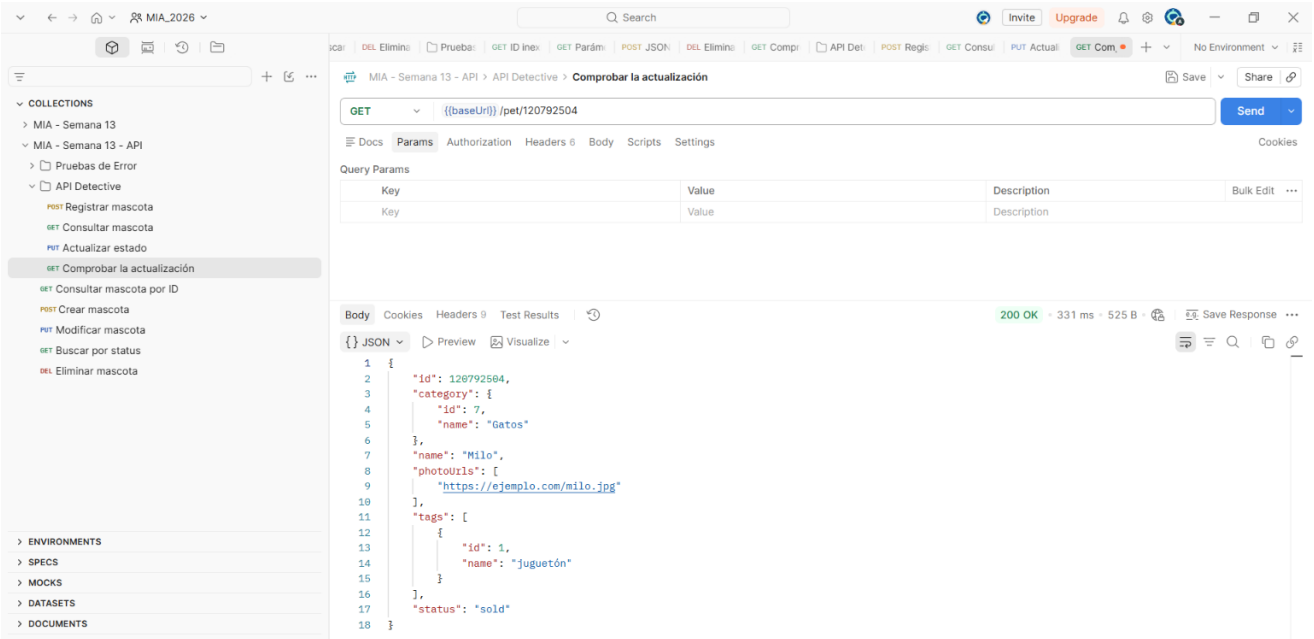
The response is a 200 status code, and the response body is a JSON object:

```
1 {
2   "id": 120792504,
3   "category": {
4     "id": 7,
5     "name": "Gatos"
6   },
7   "name": "Milo",
8   "photoUrls": [
9     "https://ejemplo.com/milo.jpg"
10  ],
11   "tags": [
12     {
13       "id": 1,
14       "name": "juguetón"
15     }
16  ],
17   "status": "available"
18 }
```

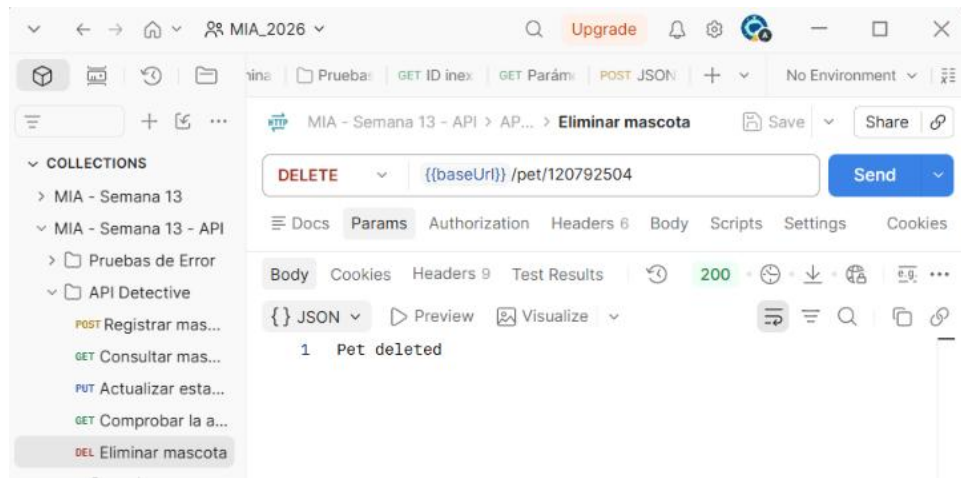
3. PUT: Actualizar estado



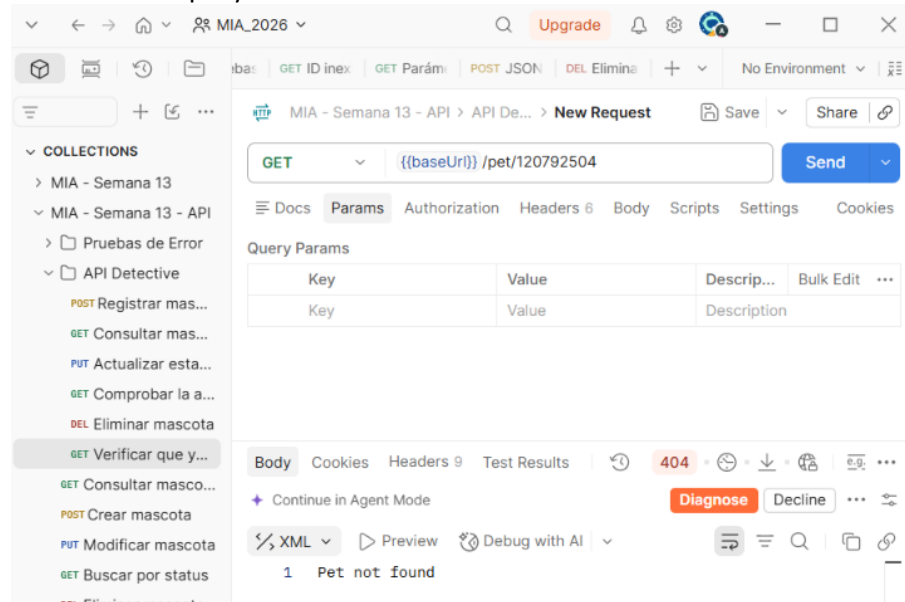
4. GET: Comprobar la actualización



5. DELETE: Eliminar mascota



6. GET: Verificar que ya no existe



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.